AI1	Dokumentacja projektu
Autor	Ewelina Kaniewska, 125129
Kierunek, rok	Informatyka, II rok, st. stacjonarne (3,5-l)
Temat projektu	*Sprzedaż słodyczy na wagę*

Spis treści

1. Wstęp 2
2. Narzędzia i technologie 3
3. Baza danych 5
4. Graficzny interfejs użytkownika 11
5. Uruchomienie aplikacji 15
6. Funkcjonalności projektu 16

1. Wstęp

Projektowana aplikacja internetowa ma na celu zarządzanie sprzedażą słodyczy na wagę. Jest to system przeznaczony zarówno dla właścicieli sklepów ze słodyczami, jak i dla ich klientów. Aplikacja umożliwia sprawne zarządzanie asortymentem, użytkownikami, dostawcami i zamówieniami. Dzięki aplikacji możliwe jest przeglądanie słodyczy i dostawców dla zwykłych użytkowników oraz użytkowników niezalogowanych. Zalogowani użytkownicy mogą zarządzać swoim kontem i zamówieniami. Administrator posiada prawo do dodawania, usuwania, aktualizowania i przeglądania wszystkich zasobów.

2. Narzędzia i technologie

Języki programowania

- **PHP:** Serwerowy język skryptowy używany do tworzenia logiki aplikacji webowej
  - Wersja: PHP 8.2.12
  - Licencja: PHP License (Open Source)
  - Dokumentacja
- **HTML:** Standardowy język znaczników używany do tworzenia struktury stron internetowych.
  - Wersja: HTML5
  - Licencja: Open (no specific license)
  - Dokumentacja
- **JavaScript:** Język skryptowy używany do tworzenia interaktywnych efektów na stronach internetowych.
  - Dokumentacja

Frameworki i biblioteki

- **Laravel:** Framework PHP używany do tworzenia aplikacji webowych.
  - Wersja: Laravel 11.7.0
  - Licencja: MIT License (Open Source)
  - Dokumentacja
- **Bootstrap:** Biblioteka CSS używana do tworzenia responsywnych i stylowych interfejsów użytkownika.
  - Wersja: Bootstrap 5
  - Licencja: MIT License (Open Source)
  - Dokumentacja

Bazy danych

- **MariaDB:** System zarządzania relacyjnymi bazami danych używany do przechowywania danych aplikacji.
  - Wersja: 10.4.32-MariaDB
  - Licencja: GPL License (Open Source)
  - Dokumentacja

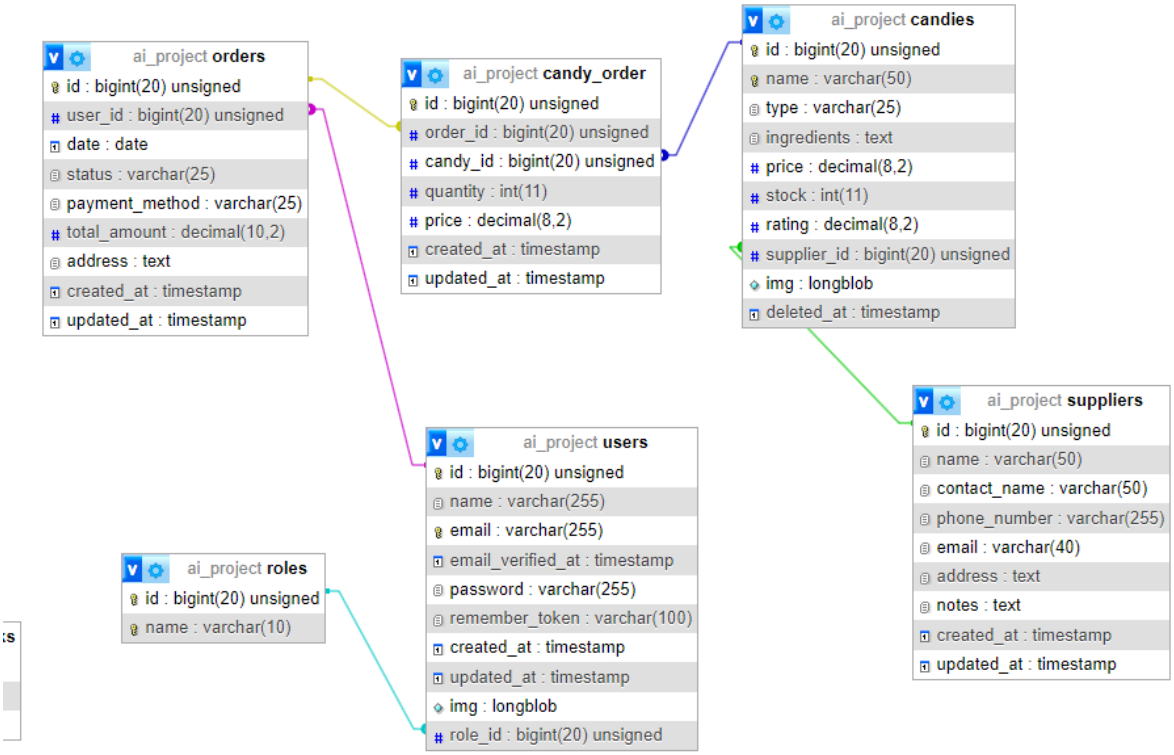
Środowiska programistyczne

- **XAMPP:** Pakiet oprogramowania zawierający Apache, MariaDB, PHP używany do lokalnego uruchamiania serwera webowego.
    - Wersja: 8.2.12
    - Licencja: GPL License (Open Source)
    - Dokumentacja
- **Visual Studio Code:** Edytor kodu źródłowego używany do programowania aplikacji.
    - Wersja: 1.89.1
    - Licencja: MIT License (Open Source)
    - Dokumentacja

Narzędzia pomocnicze

- **Composer:** Menedżer zależności dla PHP używany do zarządzania bibliotekami i pakietami.
    - Wersja 2.7.6
    - Licencja: MIT License (Open Source)
    - Dokumentacja
- **Git:** System kontroli wersji używany do zarządzania kodem źródłowym.
    - Wersja 2.45.1
    - Licencja: GPL License (Open Source)
    - Dokumentacja

3. Baza danych



Rysunek 1 Schemat ERD bazy danych

Baza danych aplikacji zarządzającej słodyczami na wagę składa się z czterech głównych tabel: candies (słodycze), suppliers (dostawcy), candy_order (słodycze_zamówienia), orders(zamówienia), users (użytkownicy) oraz roles (role).

Tabela candies

Tabela **candies** przechowuje informacje o dostępnych słodyczach. Każdy rekord w tabeli reprezentuje jeden rodzaj słodyczy.

Struktura tabeli:

- **id** (int, primary key, auto-increment): Unikalny identyfikator słodyczy.
- **name** (string, 50, unique): Nazwa słodyczy.
- **type** (string, 25): Rodzaj słodyczy.
- **ingredients** (text): Składniki słodyczy.
- **price** (decimal): Cena za kilogram.

- **stock** (int): Ilość dostępna w magazynie (w kilogramach).
- **rating** (decimal): Ocena słodyczy.
- **supplier_id** (int, foreign key): Identyfikator dostawcy, który dostarcza słodycze.
- **img** (LONGBLOB): Zdjęcie słodyczy w formacie binarnym. W przypadku tej aplikacji zdjęcia są zapisywane bezpośrednio w bazie ze względu na małą skalę projektu, jednak optymalniejszym rozwiązaniem jest przechowywanie zdjęć lokalnie i przechowywanie w bazie jedynie ich nazwy/ścieżki do nich.

id	name	type	ingredients	price	stock	rating	supplier_id	img
1	Truskawkowe landrynki	Landrynki	Cukier, syrop glukozowy, aromat truskawkowy, barwn...	23.99	200	4.20	1	[BLOB - 11.8 KB]

Rysunek 2 Przykładowe dane w tabeli candies

Tabela suppliers

Tabela **suppliers** przechowuje informacje o dostawcach słodyczy. Każdy rekord w tabeli reprezentuje jednego dostawcę.

Struktura tabeli:

- **id** (int, primary key, auto-increment): Unikalny identyfikator dostawcy.
- **name** (string, 50): Nazwa dostawcy.
- **contact_name** (string, 50): Imię i nazwisko osoby kontaktowej.
- **phone_number** (string): Numer telefonu kontaktowego.
- **email** (string, 40): Adres e-mail dostawcy.
- **address** (text): Adres dostawcy.
- **notes** (text): Dodatkowe notatki.

id	name	contact_name	phone_number	email	address	notes	created_at	updated_at
1	Runte-Bergstrom	Miss Letha Bartoletti	352-549-3316	hershel64@example.com	63204 Monica Course Hicklechester, VA 60072	Non ea unde aut voluptates reiciendis quisquam occ...	2024-01-22 01:13:40	2024-04-21 03:48:26

Rysunek 3 Przykładowe dane w tabeli suppliers

Tabela users

Tabela **users** przechowuje informacje o użytkownikach aplikacji. Każdy rekord w tabeli reprezentuje jednego użytkownika.

Struktura tabeli:

- **id** (int, primary key, auto-increment): Unikalny identyfikator użytkownika.
- **name** (string, 50): Nazwa użytkownika.
- **email** (string, 100, unique): Adres e-mail użytkownika.
- **password** (string, 255): Hasło użytkownika (przechowywane jako hash).
- **role_id** (int, foreign key): Identyfikator roli, którą posiada dany użytkownik (admin, user).
- **Img** (BLOB): zdjęcie użytkownika.

id	name	email	email_verified_at	password	remember_token	created_at	updated_at	img	role_id
1	Jan	jan@email.com	NULL	$2y$12$JrNcUGE.JnYm9ppDiYXXSeIDma3CIS.tbE1KRsivoMz...	NULL	NULL	NULL	[BLOB - 11.8 KB]	1

Rysunek 4 Przykładowe dane w tabeli users

Tabela roles

Tabela **roles** przechowuje informacje o rolach użytkowników w aplikacji. Każdy rekord w tabeli reprezentuje jedną rolę, która może być przypisana użytkownikowi. Tabela ta pozwala na łatwe zarządzanie uprawnieniami użytkowników.

Struktura tabeli:

- **id** (int, primary key, auto-increment): Unikalny identyfikator roli.
- **name** (string, 50, unique): Nazwa roli.

id	name
1	admin

Rysunek 5 Przykładowe dane w tabeli roles

Tabela orders

Tabela **orders** przechowuje informacje o zamówieniach składanych przez użytkowników aplikacji. Każdy rekord w tabeli reprezentuje jedno zamówienie.

Struktura tabeli:

- **id** (int, primary key, auto-increment): Unikalny identyfikator zamówienia.
- **user**_id (int, foreign key): Identyfikator użytkownika, który złożył zamówienie. Jest to klucz obcy odnoszący się do tabeli users. Usunięcie użytkownika powoduje kaskadowe usunięcie jego zamówień.
- **date** (date): Data złożenia zamówienia.
- **status** (string, 25): Status zamówienia.
- **payment_method** (string, 25): Metoda płatności użyta do opłacenia zamówienia.
- **total**_amount (decimal, 10, 2): Łączna kwota zamówienia.
- **address** (text): Adres dostawy zamówienia.

id	user_id	date	status	payment_method	total_amount	address	created_at	updated_at
1	5	2024-05-20	Wysłane	Karta kredytowa	569.13	99228 Fay Circle Apt. 512 South Keshaunchester, DC...	2024-04-15 11:05:36	2024-02-08 20:05:07

Rysunek 6 Przykładowe dane w tabeli orders

Tabela candy_order

Tabela **candy_order** jest tabelą łączącą w relacji wiele-do-wielu między tabelami **orders** i **candies**. Każdy rekord w tabeli reprezentuje jedno połączenie między zamówieniem a słodyczami, określając ile danego produktu znajduje się w konkretnym zamówieniu.

Struktura tabeli:

- **id** (int, primary key, auto-increment): Unikalny identyfikator rekordu.
- **order_id** (unsignedBigInteger, foreign key): Identyfikator zamówienia. Jest to klucz obcy odnoszący się do tabeli orders. Usunięcie zamówienia powoduje kaskadowe usunięcie powiązanych rekordów.
- **candy_id** (unsignedBigInteger, foreign key): Identyfikator słodyczy. Jest to klucz obcy odnoszący się do tabeli candies. Usunięcie słodyczy powoduje kaskadowe usunięcie powiązanych rekordów.
- **quantity** (integer): Ilość danego produktu w zamówieniu.
- **price** (decimal): Kwota, za którą zostały zakupione słodycze.

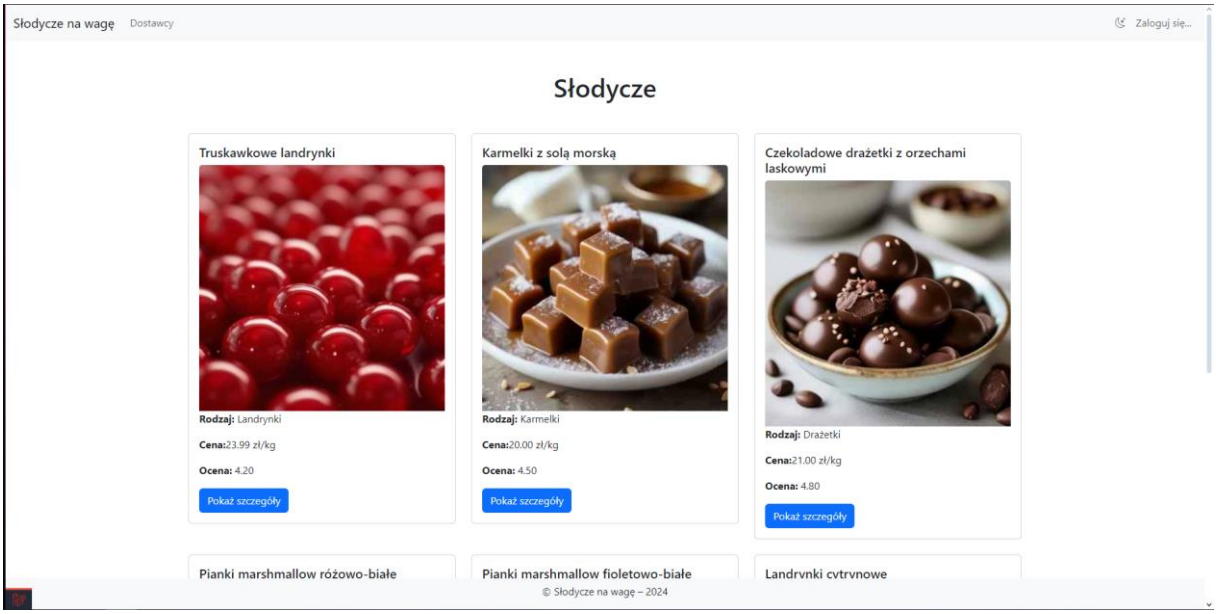
id	order_id	candy_id	quantity	price
1	1	2	5	21.11

Rysunek 7 Przykładowe dane w tabeli candy_order

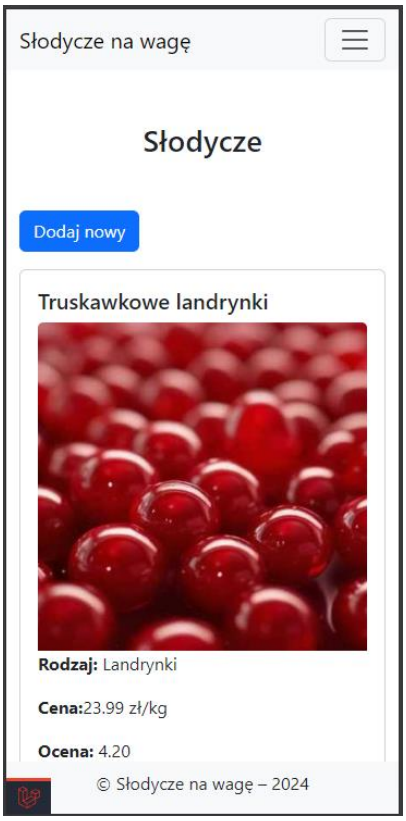
Baza danych aplikacji zarządzającej słodyczami na wagę składa się z sześciu głównych tabel: **candies**, **suppliers**, **users**, **roles**, **orders**, oraz **candy_order.** Tabele te są ze sobą powiązane relacjami, co umożliwia efektywne zarządzanie informacjami o słodyczach, ich dostawcach, użytkownikach aplikacji, zamówieniach oraz rolach użytkowników. Tabela **candies** przechowuje szczegóły dotyczące każdego rodzaju słodyczy. Tabela **suppliers** zawiera informacje o dostawcach, którzy dostarczają słodycze. Tabela **users** przechowuje dane o użytkownikach aplikacji, w tym ich nazwę, adres e-mail, hasło oraz przypisaną rolę. Tabela **roles** zawiera informacje o różnych rolach użytkowników, takich jak administratorzy i zwykli użytkownicy. Tabela **orders** gromadzi informacje o zamówieniach składanych przez użytkowników, w tym datę zamówienia, status, metodę płatności, łączną kwotę oraz adres dostawy. Tabela **candy_order** jest tabelą pośredniczącą (łączącą) w relacji wiele-do-wielu między tabelami **orders** i **candies**, przechowującą informacje o ilości danego produktu w konkretnym zamówieniu. Relacje między tabelami pozwalają na powiązanie słodyczy z ich dostawcami, określenie ról użytkowników, co umożliwia kontrolę dostępu i uprawnień w aplikacji, składanie zamówień przez użytkowników oraz śledzenie szczegółów każdego zamówienia oraz zarządzanie ilością słodyczy w magazynie w oparciu o złożone zamówienia.

4. Graficzny interfejs użytkownika

Uruchamiając aplikację użytkownik widzi pierwszy - główny - z dostępnych widoków, który przedstawia dostępne słodycze w formie kart.

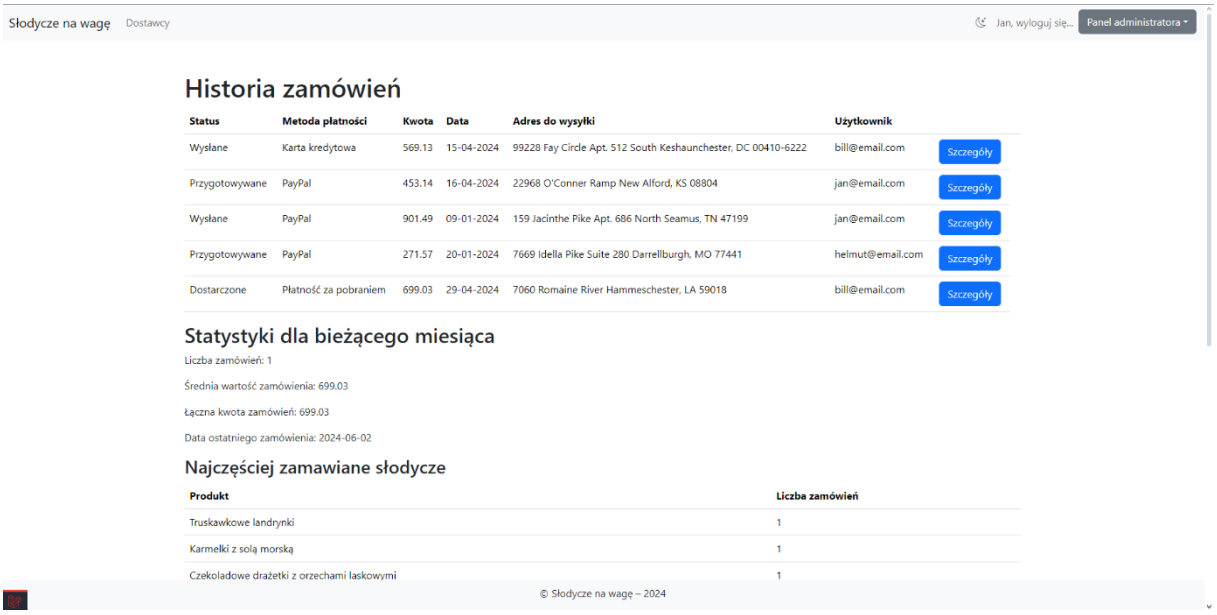


Rysunek 8 Główny widok aplikacji

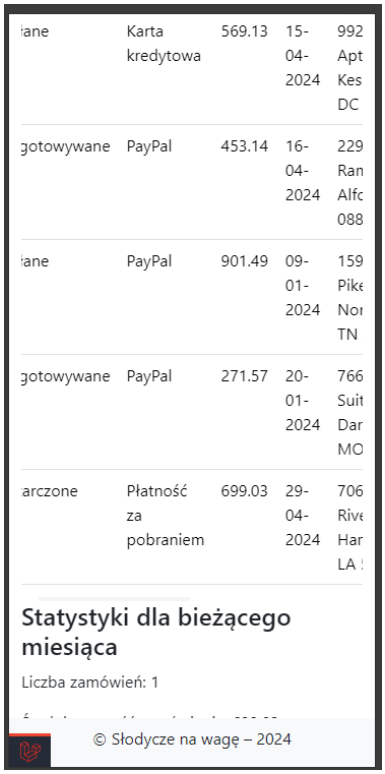


Rysunek 9 Wygląd aplikacji na urządzeniu mobilnym

Po kliknięciu w opcję zarządzania zamówieniami w rozwijanym menu „Panel administratora” administrator widzi wylistowane wszystkie zamówienia i statystyki z obecnego miesiąca.

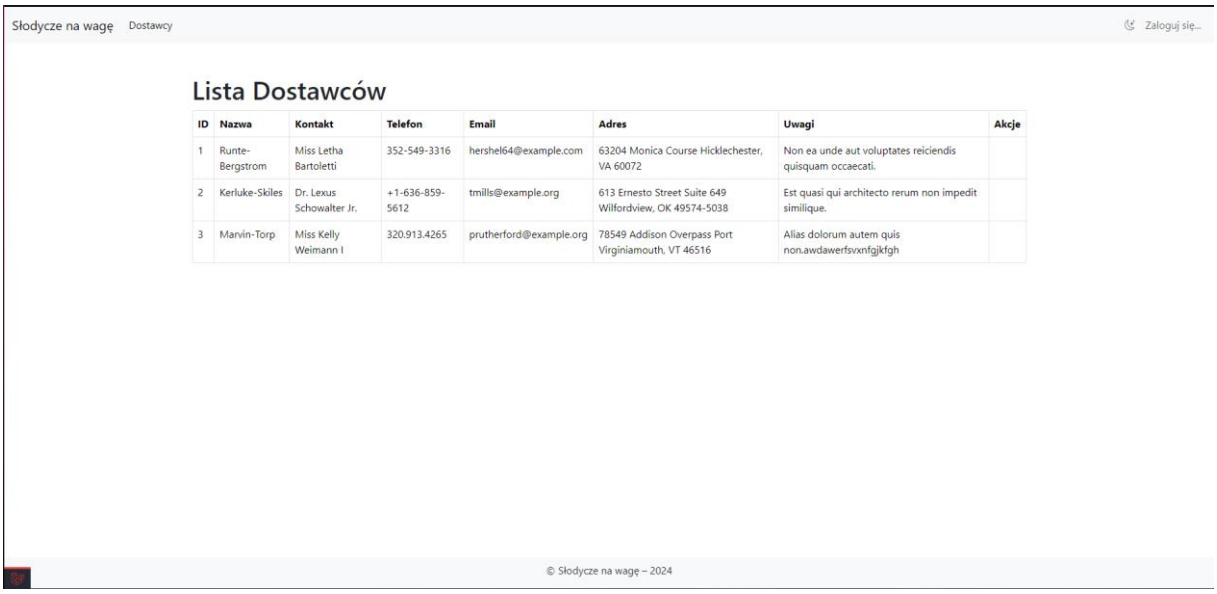


Rysunek 10 Widok zarządzania zamówieniami



Rysunek 11 Widok zarządzania zamówieniami na urządzeniu mobilnym

Klikając w segment ‘Dostawcy’ w lewym górnym rogu strony użytkownik przechodzi do listy dostawców słodyczy

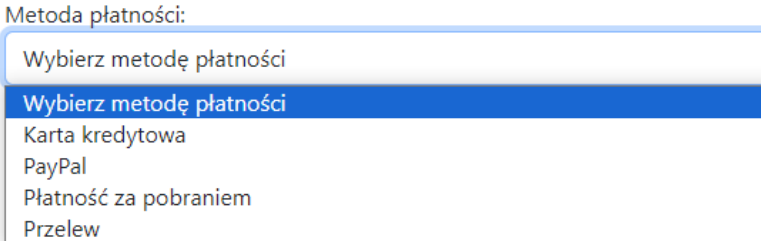


ID	Nazwa	Kontakt	Telefon	Email	Adres	Uwagi	Akcje
1	Runte-Bergstrom	Miss Letha Bartoletti	352-549-3316	hershel64@example.com	63204 Monica Course Hicklechester, VA 60072	Non ea unde aut voluptates reiciendis quisquam occaecati.	
2	Kerluke-Skiles	Dr. Lexus Schowalter Jr.	+1-636-859-5612	tmills@example.org	613 Ernesto Street Suite 649 Wilfordview, OK 49574-5038	Est quasi qui architecto rerum non impedit similique.	
3	Marvin-Torp	Miss Kelly Weimann I	320.913.4265	prutherford@example.org	78549 Addison Overpass Port Virginiamouth, VT 46516	Alias dolorum autem quis non.awdawerfsvxnfgjkfgh	

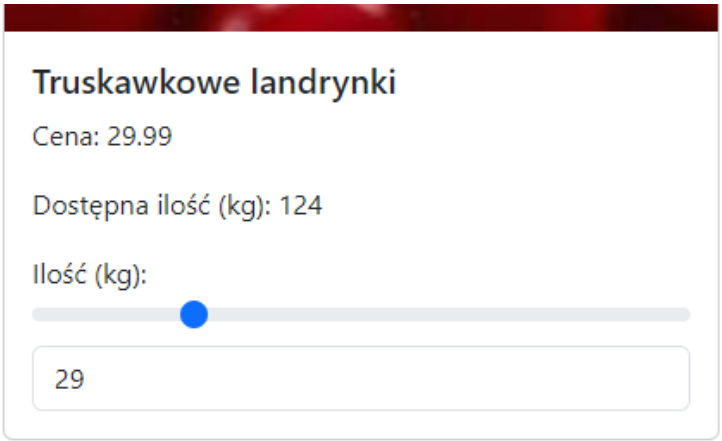
Rysunek 12 Lista dostawców

W aplikacji w wielu widokach pojawiają się różnie kontrolki interfejsu, np:

Rysunek 13 Wyłączanie / włączanie trybu ciemnego



Rysunek 14 Lista rozwijana w formularzu



Rysunek 15 Suwak do określania ilości zamówionego produktu

5. Uruchomienie aplikacji

Do uruchomienia aplikacji potrzebne jest zainstalowanie odpowiedniego oprogramowania

- XAMPP
- Composer

Instalację programu xampp należy przeprowadzić krok po kroku za pomocą instalatora (na dysku systemowym). Po zainstalowaniu należy dodać zmienną środowiskową

```
C:\xampp\php
```

Podobnie zainstalować composer.

Następnie należy włączyć poprzez xamppa Apache i MySQL.

Aby przygotować projekt do działania należy uruchomić skrypt start.bat umieszczony w katalogu projektu.

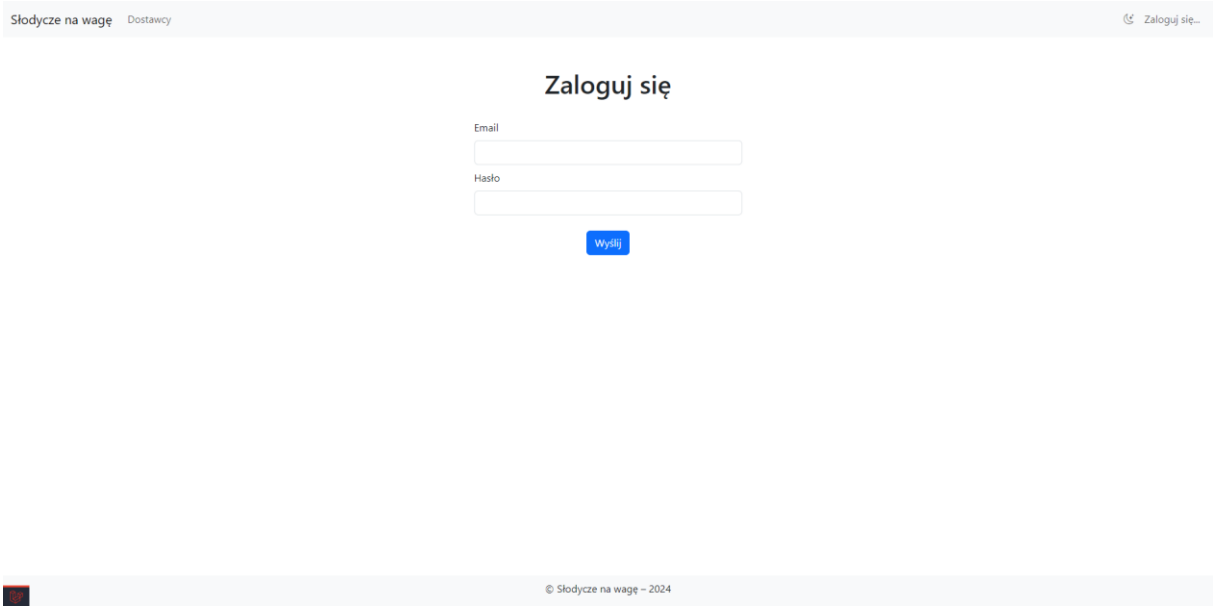
Teraz baza danych jest gotowa do korzystania z aplikacji.
Należy jeszcze tylko uruchomić serwer komendą
php artisan serve
w katalogu aplikacji.

Aby dostać się do głównego widoku projektu należy w pasku adresu przeglądarki wpisać *http://127.0.0.1:8000*.

6. Funkcjonalności projektu

- Logowanie

Do formularza logowania prowadzi odnośnik w prawym górnym rogu.



Rysunek 16 Formularz logowania

Dane aministratora:

Email: jan@email.com

Hasło: 1234

Logowanie jest obsługiwane przez kontroler AuthController. Kontroler ten zawiera metody odpowiadające za wyświetlanie formularza logowania, obsługę procesu logowania, a także wylogowywanie użytkowników.

```
public function login()
{
    if (Auth::check()) {
        return redirect()->route('candies.index');
    }
    return view('auth.login');
}
```

Rysunek 17 Metoda login

Cel: Wyświetlenie formularza logowania.

Działanie: Metoda sprawdza, czy użytkownik jest już zalogowany za pomocą Auth::check().

Jeśli użytkownik jest zalogowany, następuje przekierowanie do strony głównej aplikacji (candies.index).

Jeśli użytkownik nie jest zalogowany, wyświetlany jest widok formularza logowania (auth.login).

```
public function authenticate(Request $request)
{
    $credentials = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required'],
    ]);

    if (Auth::attempt($credentials)) {
        $request->session()->regenerate();
        return redirect()->route('candies.index');
    }

    return back()->withErrors([
        'email' => 'The provided credentials do not match our records.',
    ])->onlyInput('email');
}
```

Rysunek 18 Metoda authenticate

Cel: Obsługa próby logowania.

Działanie:

Metoda najpierw waliduje dane wejściowe, sprawdzając, czy pola email i password są obecne i poprawne.

Następnie używa Auth::attempt($credentials), aby spróbować zalogować użytkownika za pomocą podanych danych.

Jeśli dane logowania są poprawne sesja użytkownika jest regenerowana.

Następuje przekierowanie do strony głównej aplikacji (candies.index).

Jeśli dane logowania są niepoprawne użytkownik jest przekierowany z powrotem do formularza logowania z komunikatem o błędzie.

```php
public function logout(Request $request)
{
    Auth::logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();
    return redirect()->route('candies.index');
}
```

Rysunek 19 Metoda logout

Cel: Obsługa procesu wylogowywania użytkownika.

Działanie:

Metoda wylogowuje użytkownika za pomocą Auth::logout().

Następnie sesja użytkownika jest unieważniana ($request->session()->invalidate()) oraz generowany jest nowy token CSRF ($request->session()->regenerateToken()), co dodatkowo zabezpiecza aplikację.

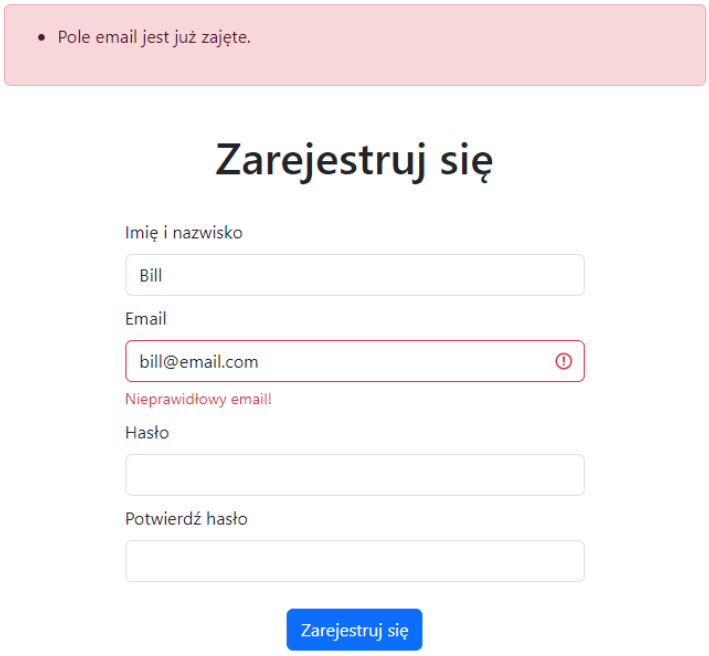
Na koniec następuje przekierowanie do strony głównej aplikacji (candies.index).

- Rejestracja

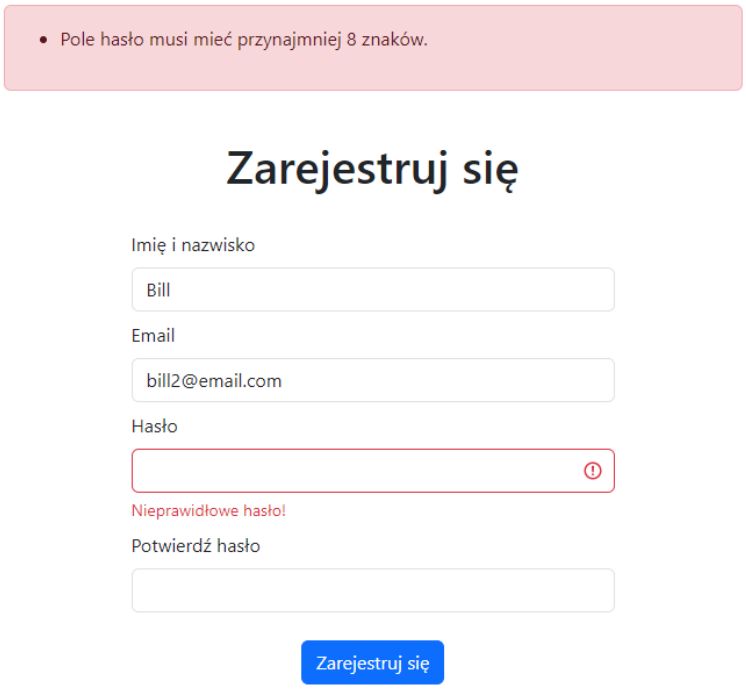
```php
public function register(Request $request)
{
    $request->validate([
        'name' => ['required', 'string', 'max:50'],
        'email' => ['required', 'string', 'email', 'max:100', 'unique:users'],
        'password' => ['required', 'string', 'min:8', 'confirmed'],
    ]);

    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
        'role_id' => 2,
    ]);

    Auth::login($user);
    $request->session()->regenerate();
    return redirect()->route('candies.index');
}
```

Rysunek 20 Metoda register w AutController

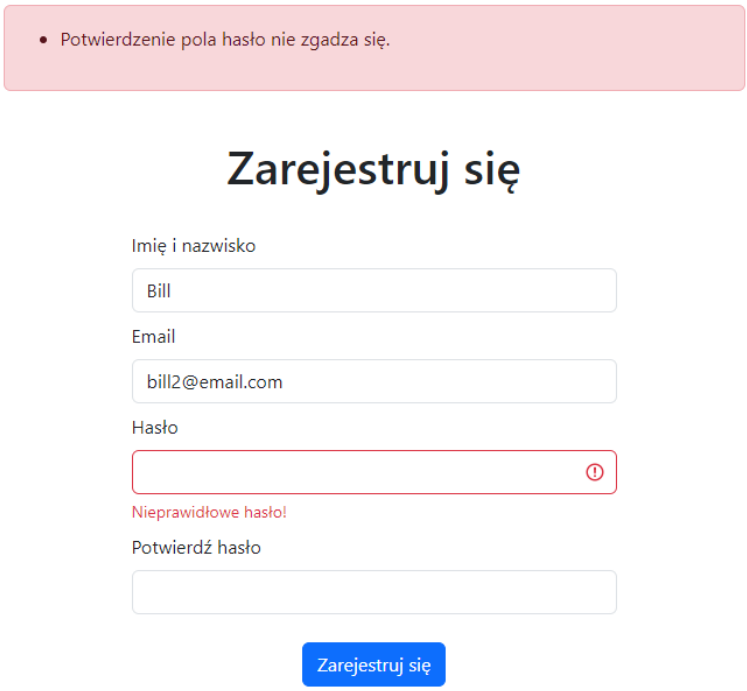
Funkcja register służy do rejestracji nowego użytkownika w systemie. Najpierw sprawdza poprawność przesłanych danych, takich jak nazwa, adres e-mail i hasło. Jeśli dane są poprawne, tworzony jest nowy rekord w bazie danych zawierający informacje o użytkowniku, takie jak nazwa, e-mail (który musi być unikalny) i hasło. Rola jest przypisywana 'na sztywno' jako użytkownik. Nowy użytkownik jest również automatycznie logowany do systemu, a jego sesja jest regenerowana dla bezpieczeństwa. Po zakończeniu procesu rejestracji użytkownik jest przekierowywany na stronę główną aplikacji.



Rysunek 21 Przypadek, kiedy email już istnieje w bazie danych



Rysunek 22 Przypadek, kiedy hasło jest krótsze niż 8 znaków



Rysunek 23 Przypadek, kiedy hasła różnią się od siebie

- CRUD

I. CREATE

Tworzenie nowego produktu jest realizowane przez metody create i store kontrolera CandyConroller. Tylko administrator może dodawać nowe zasoby.

```
@can('is-admin')
        <a href="{{ route('candies.edit', $candy->id) }}" class="btn btn-warning">Edytuj</a>
        <form action="{{ route('candies.destroy', $candy->id) }}" method="POST" class="d-inline">
            @csrf
            @method('DELETE')
            <button type="submit" class="btn btn-danger">Usuń</button>
        </form>
    @endcan
```

Rysunek 24 Ukrycie przycisków dla zwykłych użytkowników

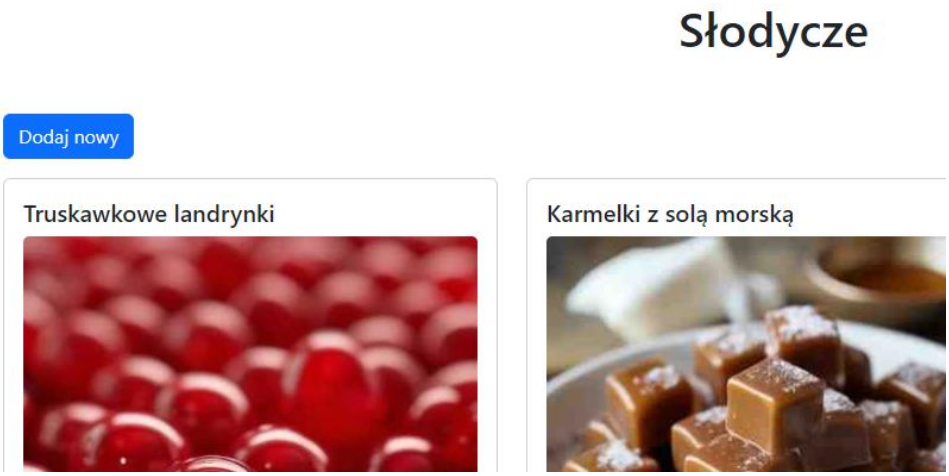
Ponadto zostało zastosowane zabezpieczenie, dzięki któremu tylko administrator może wykonać akcję dodawania produktu.

```
class StoreCandyRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return Gate::allows('is-admin');
    }
```

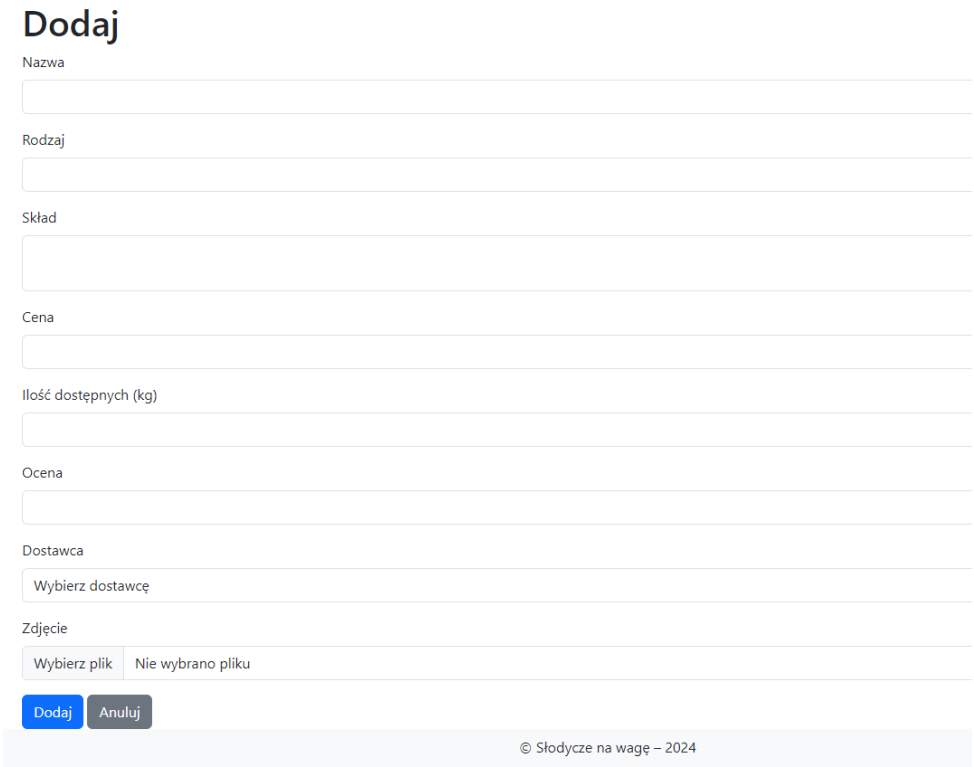
Rysunek 25 Zabezpieczenie przed dodaniem zasobu przez nieupoważnionego użytkownika

```
public function create()
{
    if (! Gate::allows('is-admin')) {
        abort(403);
    }

    $suppliers = Supplier::all();
    return view('candies.create', ['suppliers' => $suppliers]);
}
```

Rysunek 26 Zabezpieczenie przed wyświetleniem formularza dodawania



Rysunek 27 Administrator ma dostępny przycisk dodawania produktu



Rysunek 28 Formularz dodawania produktu

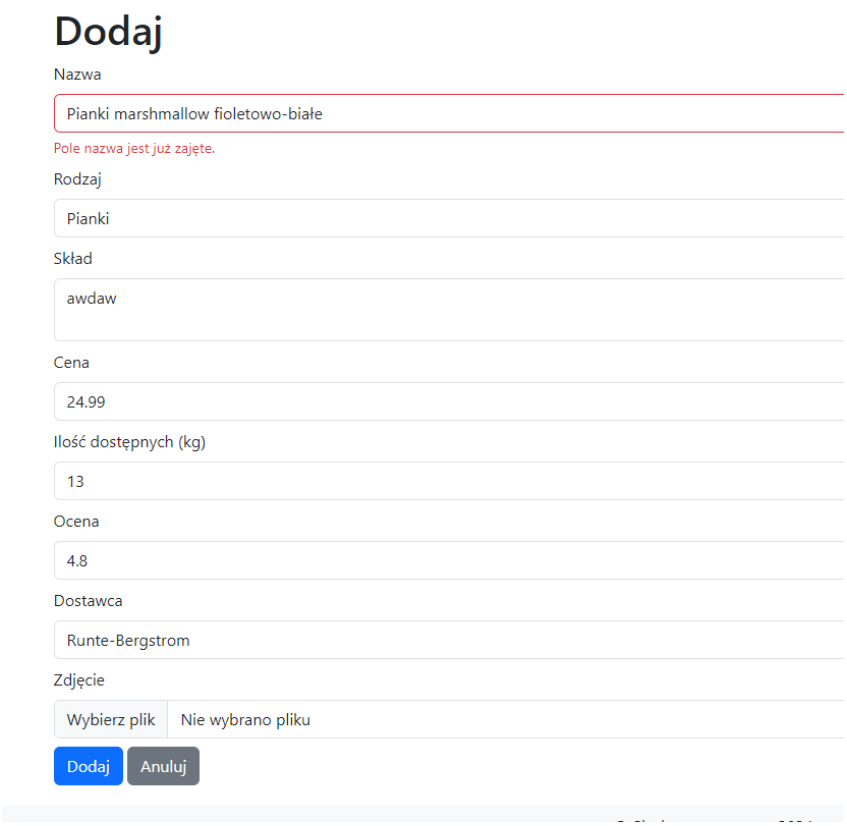
```php
public function rules(): array
{
    return [
        'name' => 'required|unique:candies,name,|max:50',
        'type' => 'required|string|max:25',
        'ingredients' => 'required|string',
        'price' => 'required|numeric|between:0,9999.99',
        'stock' => 'required|integer|min:0',
        'rating' =>'required|numeric|between:0,5'
    ];
}
```

Rysunek 29 Walidacja danych

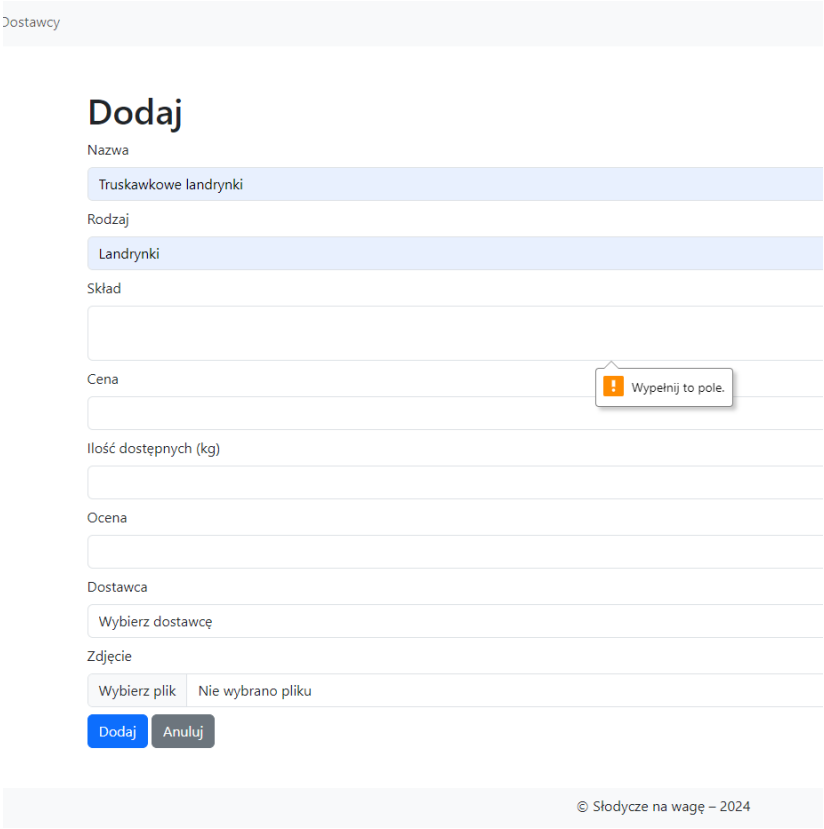
Dane są walidowane za pomocą metody rules() w StoreCandyRequest. Oprócz tego walidacja występuje też w formularzach.

```html
<input id="name" name="name" type="text" required maxlength="50"
```

Rysunek 30 Walidacja danych w formularzu



Rysunek 31 Przykład podania nazwy, która już istnieje

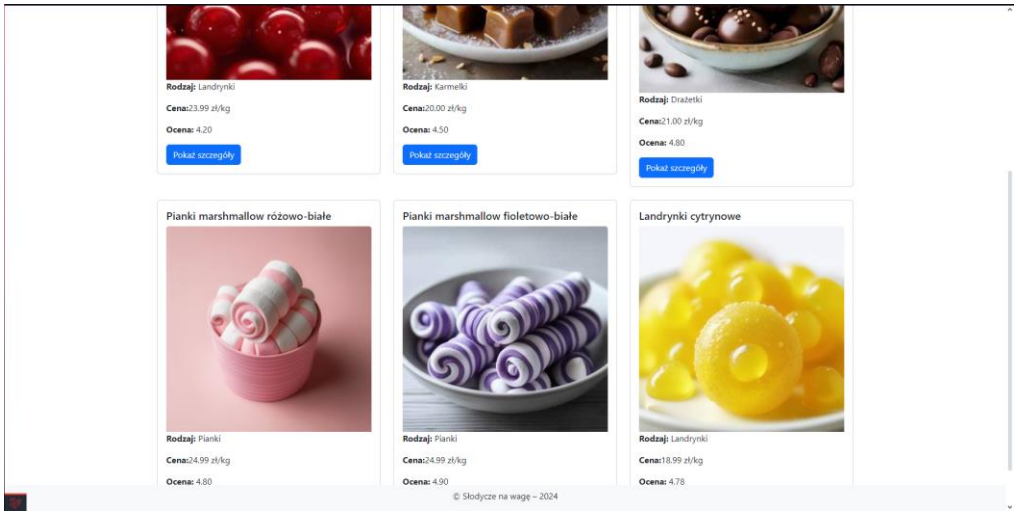


Rysunek 32 Przykład, kiedy pole pozostanie puste

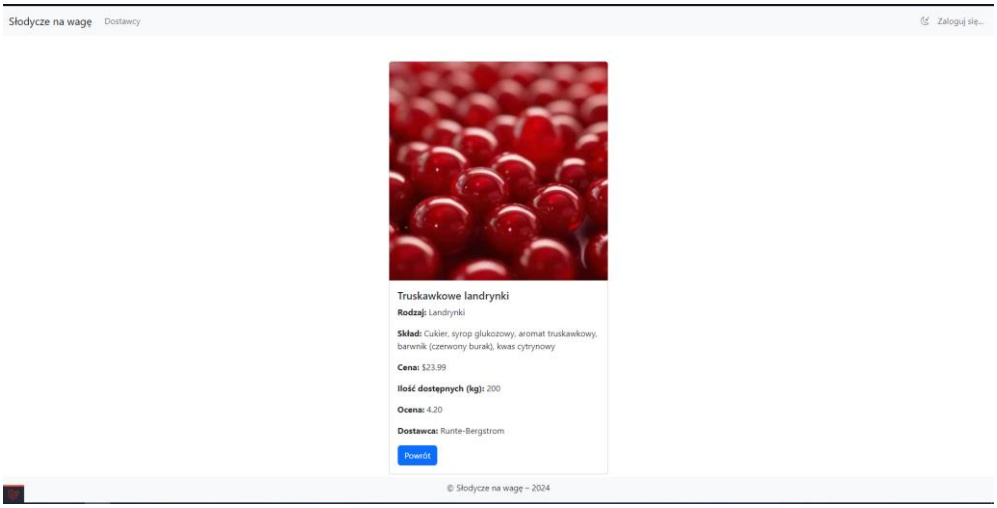
W przypadku, kiedy użytkownik, który nie jest adminem spróbuje wejść na podstronę z formularzem dodawania produktu zostanie przekierowany na podstronę logowania. Kiedy spróbuje to zrobić niezalogowany użytkownik zostanie wyświetlony komunikat błędu autoryzacji (wykorzystano middleware).

II. READ

Przeglądanie zasobów jest realizowane za pomocą metod index() i show($id) kontrolera CandyController. Każdy użytkownik (również niezalogowany) może wyświetlać wszystkie zasoby



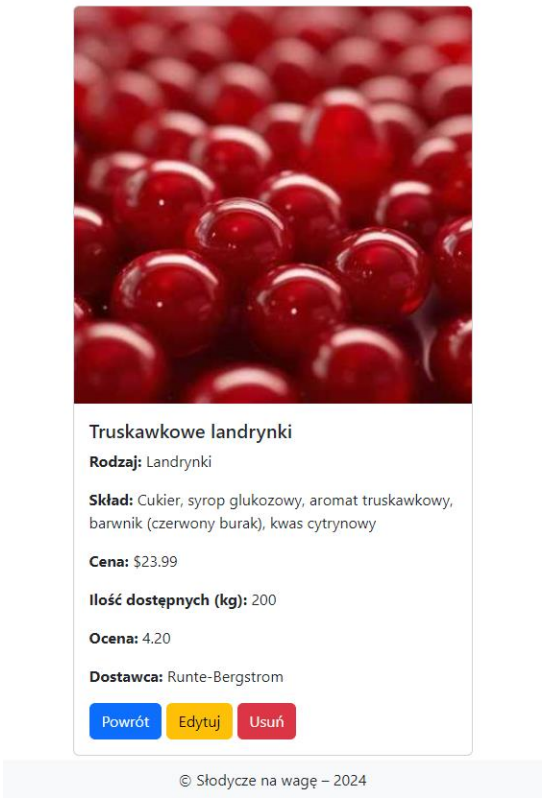
Rysunek 33 Przeglądanie ogólnodostępnych zasobów



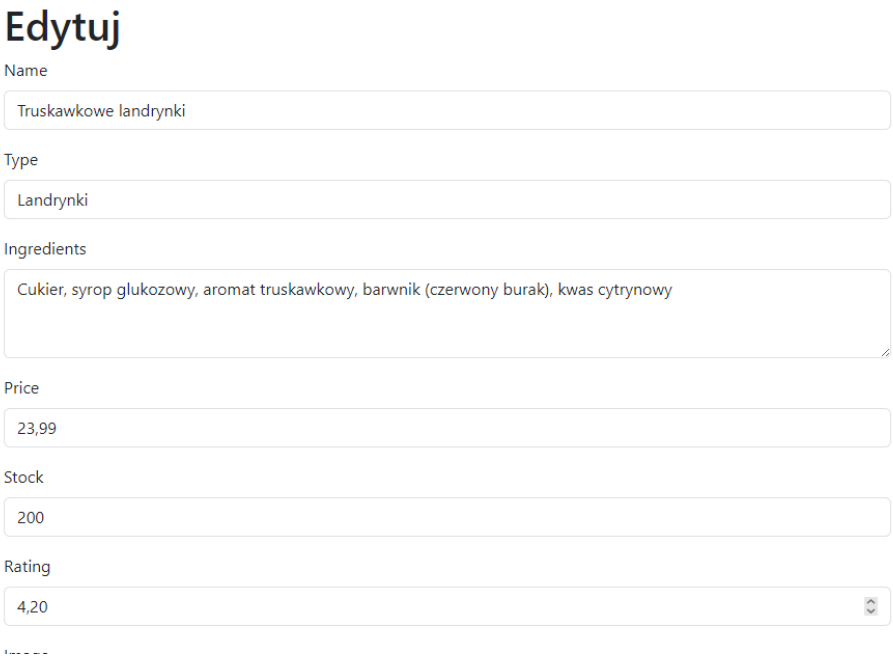
Rysunek 34 Wyświetlanie ogólnie dostępnego zasobu

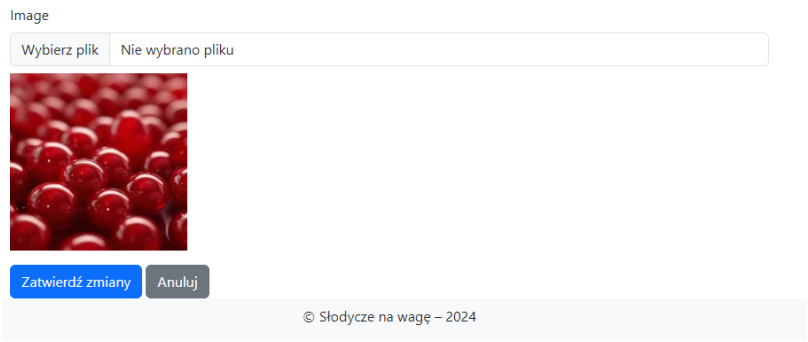
III. UPDATE

Aktualizowanie danych jest realizowane za pomocą funkcji edit i update. Tylko administrator może dokonywać aktualizacji.



Rysunek 35 Przyciski dostępne dla administratora





Rysunek 36 Formularz edycji

Zastosowano tutaj takie same mechanizmy zabezpieczające jak w przypadku dodawania zasobu: użytkownik, który nie jest adminem nie może zobaczyć przycisku ani wejść na podstronę z formularzem edycji.

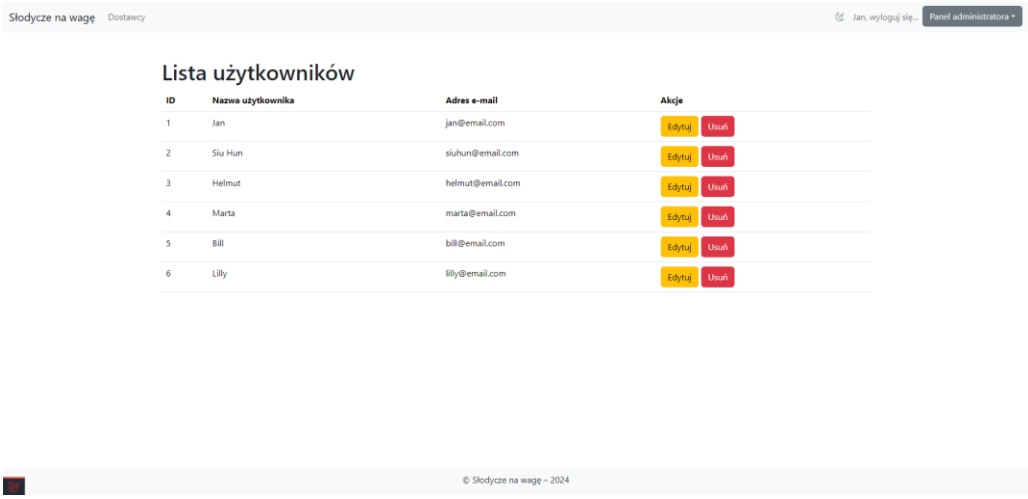
IV. DELETE

Za usuwanie danych odpowiedzialna jest metoda delete kontrolera CandyController. Tylko admin może usunąć dane. Zabezpieczenia są takie same jak w przypadku Create i Update.

W taki sam sposób została przeprowadzona implementacja CRUD dla pozostałych tabel.

- **Zarządzanie użytkownikami przez administratora**

Administrator może wyświetlić listę użytkowników



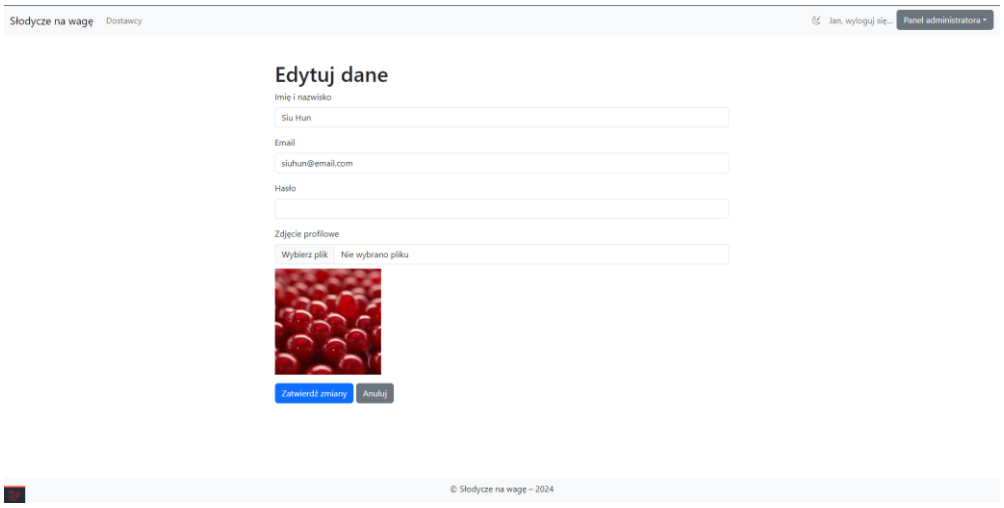
Rysunek 37 Widok zarządzania użytkownikami

Ten widok może wyświetlić tylko administrator, dzięki middleware, które sprawdza jaką rolę ma przypisaną użytkownik.

```
public function handle(Request $request, Closure $next): Response
{
    if (auth()->check()) {
        if(\Auth::user()->role_id != 1)
    {
        abort(401, 'Brak uprawnień');
    }
    return $next($request);
    } else {
        return redirect()->route('login');
    }

}
```

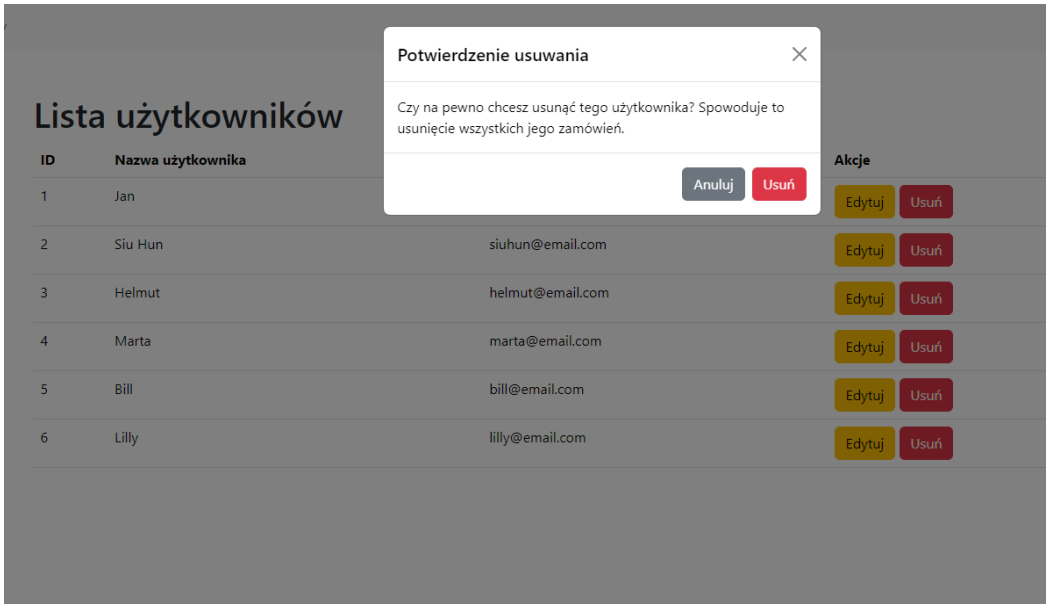
Rysunek 38 Middleware

Administrator może edytować dane użytkownika.

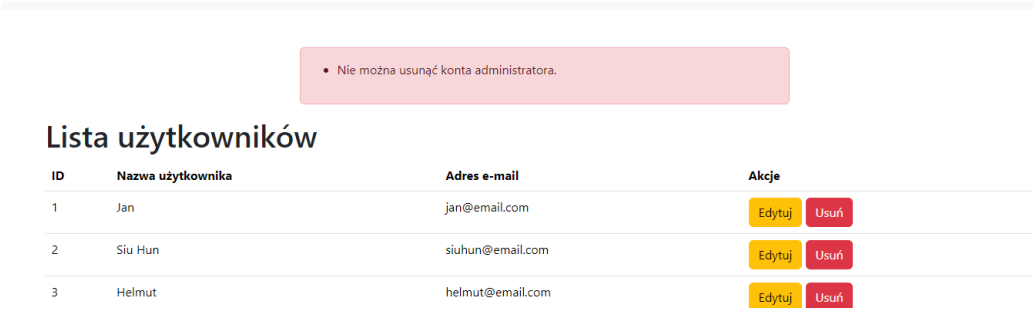


Rysunek 39 Edycja danych użytkownika

Może również usunąć użytkownika. Kiedy podejmuje próbę usunięcia konta o roli administratora, pojawia się komunikat błędu o zakazie wykonania takiej akcji. Usuwanie użytkownika należy zatwierdzić ponownie.



Rysunek 40 Usuwanie konta użytkownika

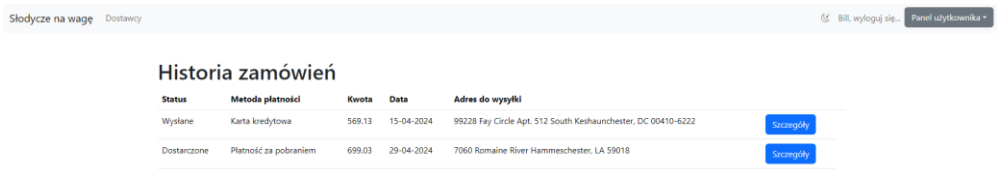


Rysunek 41 Próba usunięcia konta administratora

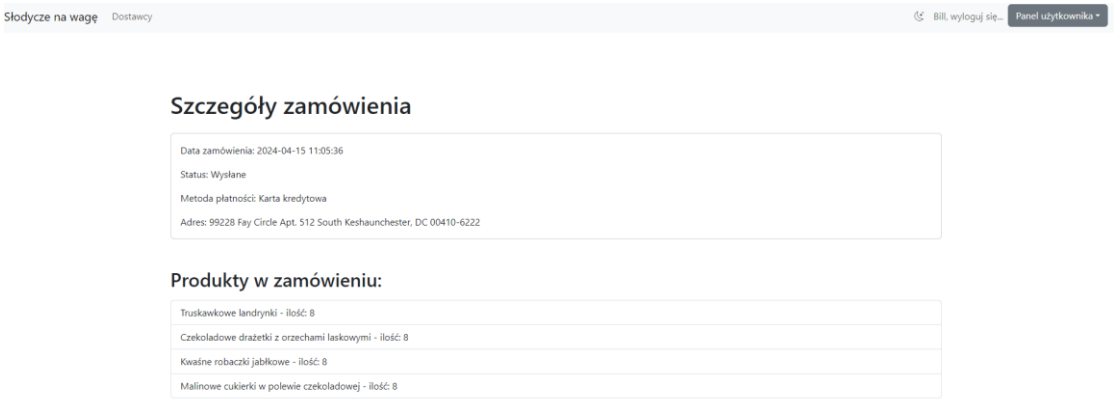
Autoryzacja użytkownika odbywa się również przez midleware.

- **Zarządzanie swoimi zasobami przez użytkownika aplikacji**

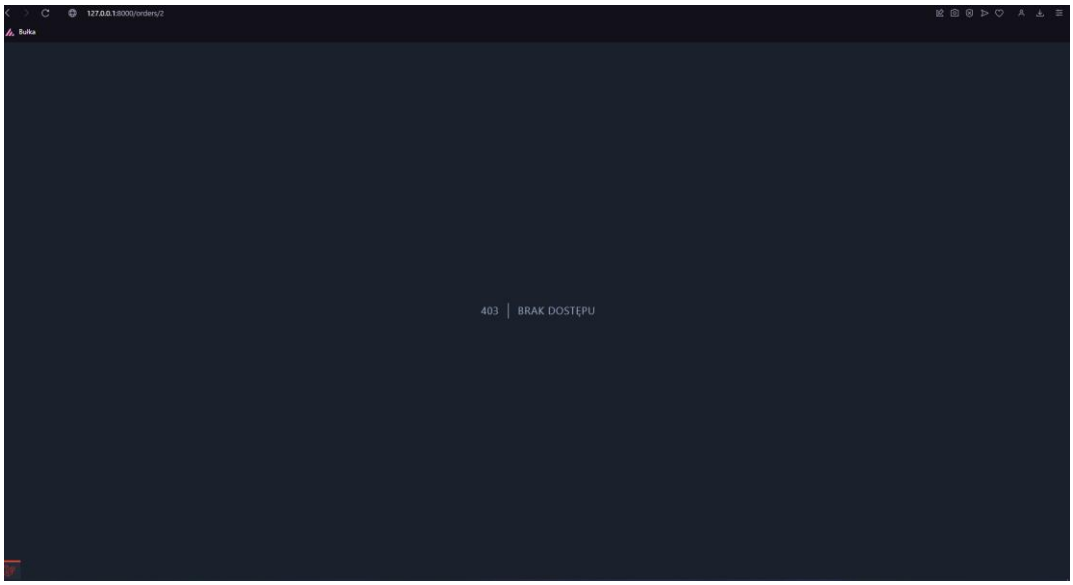
Użytkownik może wyświetlić swoje zamówienia. Próba wyświetlenia zamówień innego użytkownika zakończy się niepowodzeniem.



Rysunek 42 Historia zamówień w panelu użytkownika



Rysunek 43 Wyświetlenie szczegółów zamówienia



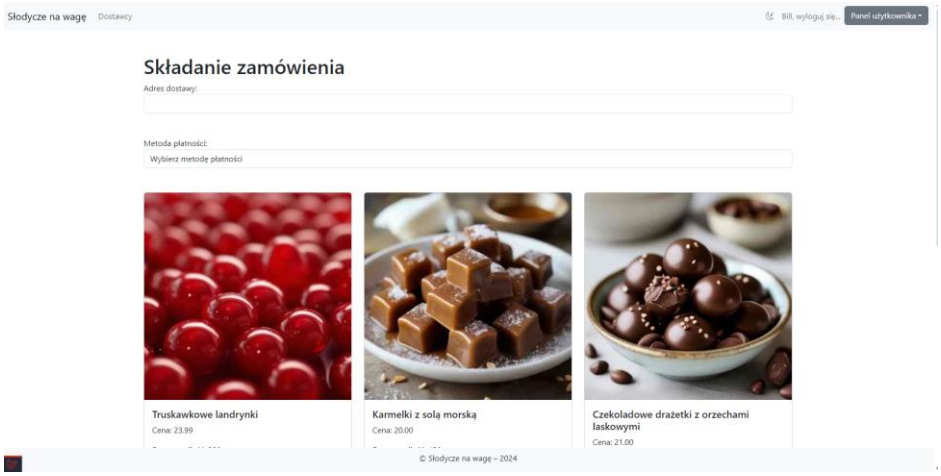
Rysunek 44 Próba wyświetlenia zamówienia innego użytkownika

Za to zabezpieczenie odpowiada instrukcja warunkowa, która sprawdza czy id zalogowanego użytkownika jest takie samo jak id użytkownika, którego zamówienie próbowano wyświetlić.

```
if (Auth::id() !== $order->user_id) {
    abort(403, 'Brak dostępu');
}
```

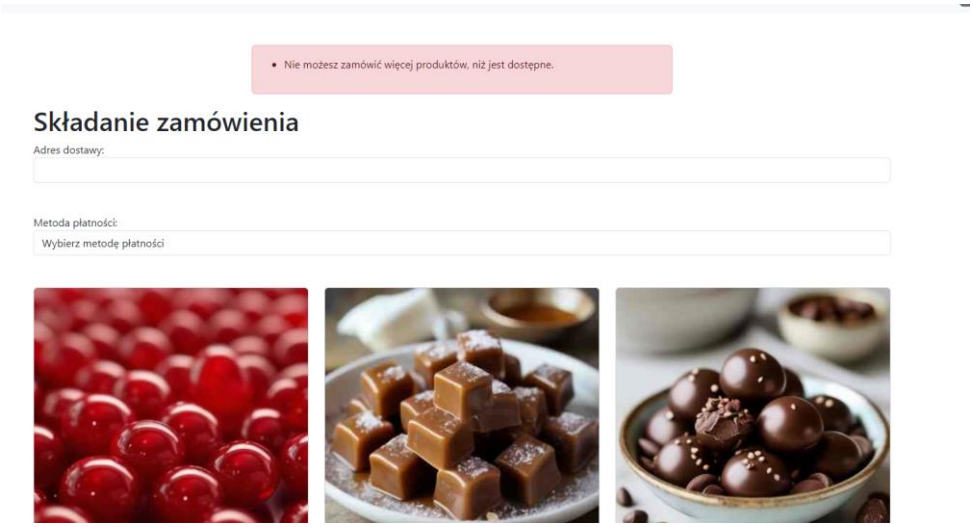
Rysunek 45 Instrukcja odpowiedzialna za autoryzację wyświetlenia zamówienia

Użytkownik może złożyć nowe zamówienie. Walidacja danych odbywa się na podobnej zasadzie jak przy operacjach na słodyczach.



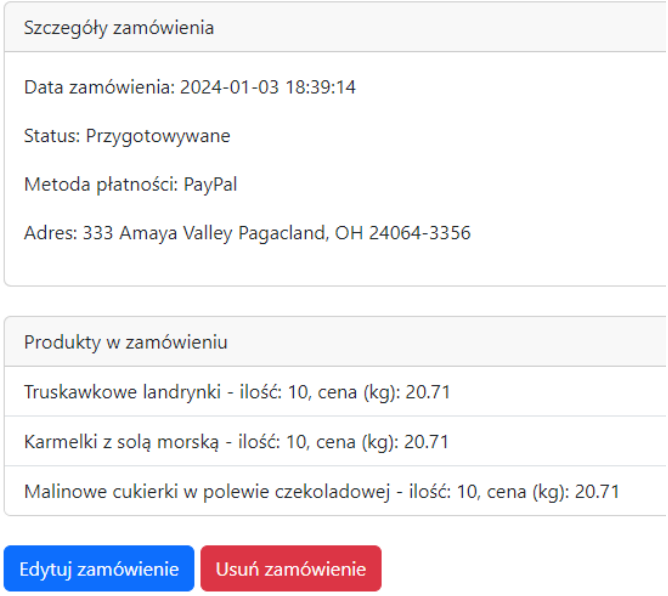
Rysunek 46 Formularz składania zamówienia

Przy próbie zamówienia większej ilości niż jest dostępna użytkownik zobaczy odpowiedni komunikat błędu.



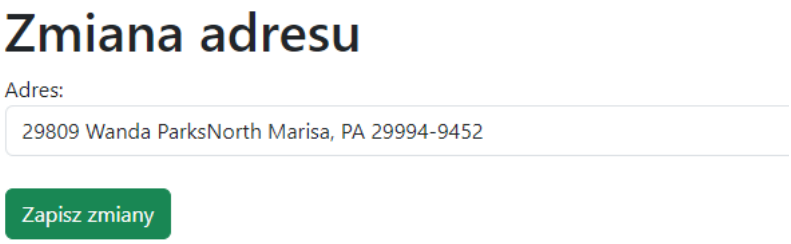
Rysunek 47 Walidacja

Jeśli zamówienie nie zostało jeszcze wysłane, użytkownik może je usunąć lub edytować.



Rysunek 48 Widoczne przyciski usuwania i edycji

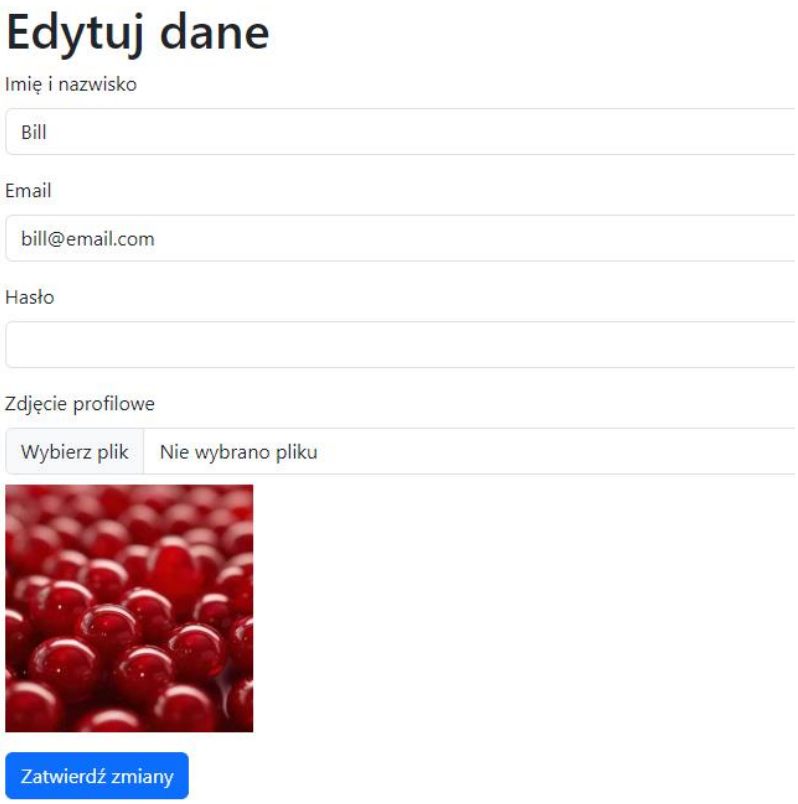
Użytkownik może edytować tylko adres dostawy.



Rysunek 49 Zmiana adresu dostawy

- **Zarządzanie swoimi danymi przez użytkownika aplikacji**

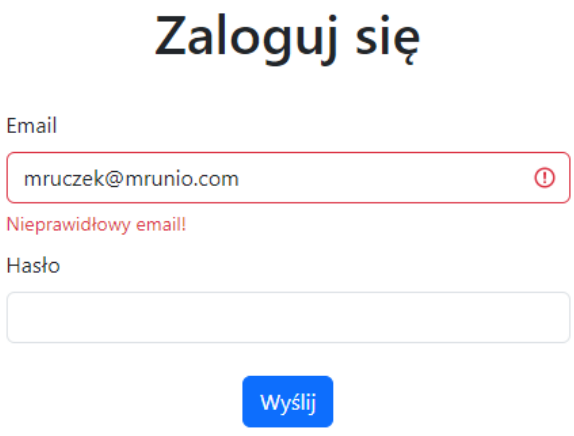
  Użytkownik może edytować informacje o swoim koncie.



Rysunek 50 Formularz edycji konta

- **Proces logowania się użytkowników aplikacji**

  Z formularza logowania dane są przesyłane do metody authenticate kontrolera AuthController. Następnie przechodzą walidację i następuje próba zalogowania wykorzystująca metodę attempt. W przypadku znalezienia opowiadającego rekordu w bazie danych użytkownik zostaje zalogowany i przeniesiony do głównego widoku aplikacji. W przypadku, kiedy dane są niepoprawne pojawia się odpowiedni komunikat błędu.



Rysunek 51 Błędne dane logowania

- **Statystyki**

Aplikacja umożliwia użytkownikowi wyświetlenie statystyk sprzedaży.

Statystyki dla bieżącego miesiąca

Liczba zamówień: 1

Średnia wartość zamówienia: 699.03

Łączna kwota zamówień: 699.03

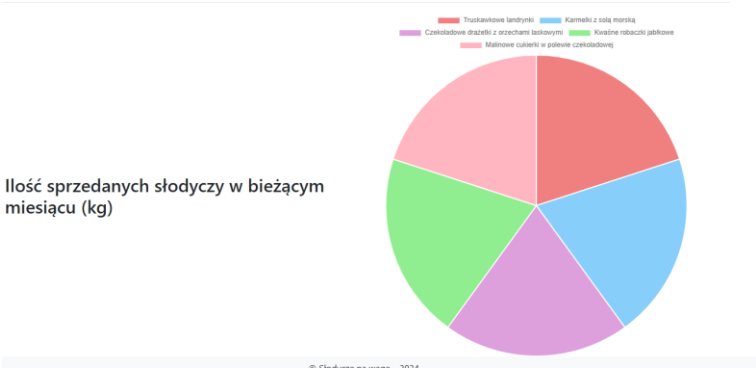
Data ostatniego zamówienia: 2024-06-02

Najczęściej zamawiane słodycze

Produkt	Liczba zamówień
Truskawkowe landrynki	1
Karmelki z solą morską	1
Czekoladowe drażetki z orzechami laskowymi	1
Kwaśne robaczki jabłkowe	1
Malinowe cukierki w polewie czekoladowej	1

Rysunek 52 Statystyki

Administrator może także sprawdzić ilość sprzedanych słodyczy w danym miesiącu na wykresie kołowym stworzonym przy pomocy biblioteki chart.js.

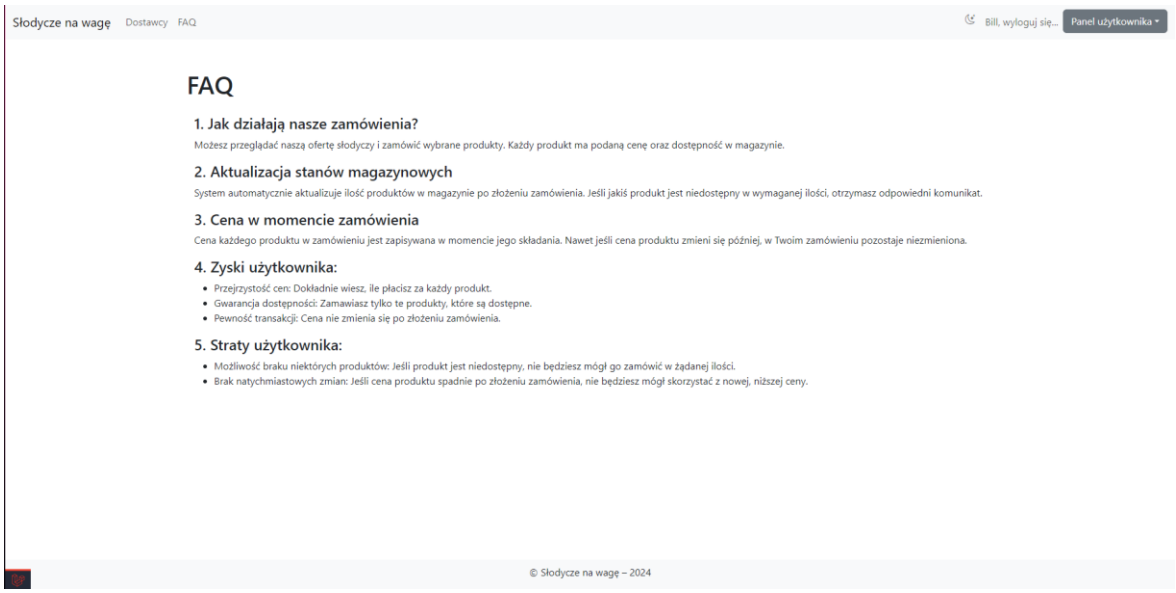


Rysunek 53 Wykres kołowy

- Logika biznesowa

  Proces składania zamówień:

  - **Wybór produktów**: Użytkownik ma możliwość przeglądania dostępnych produktów w sklepie, zapoznając się z ich opisami i cenami.
  - **Określenie ilości**: Po wybraniu interesującego produktu, użytkownik może zdefiniować żądaną ilość poprzez wpisanie liczby lub przesunięcie suwaka.
  - **Wybór metody dostawy**: Klient może wybrać preferowaną metodę dostawy spośród dostępnych opcji.
  - **Podanie adresu dostawy**: Użytkownik ma możliwość wprowadzenia adresu dostawy, który może później zmienić, jeśli zamówienie nie będzie wysłane.
  - **Wyświetlenie ceny**: Wszystkie koszty są transparentnie wyświetlane w zamówieniu.
  - **Sprawdzenie dostępności produktu**: Przed zatwierdzeniem zamówienia, system sprawdza dostępność każdego produktu w magazynie. Zamówienie jest akceptowane tylko w przypadku, gdy wszystkie produkty są dostępne.
  - **Aktualizacja stanu magazynowego**: Po złożeniu zamówienia, system automatycznie aktualizuje stan magazynowy, aby odzwierciedlić zakupione produkty.



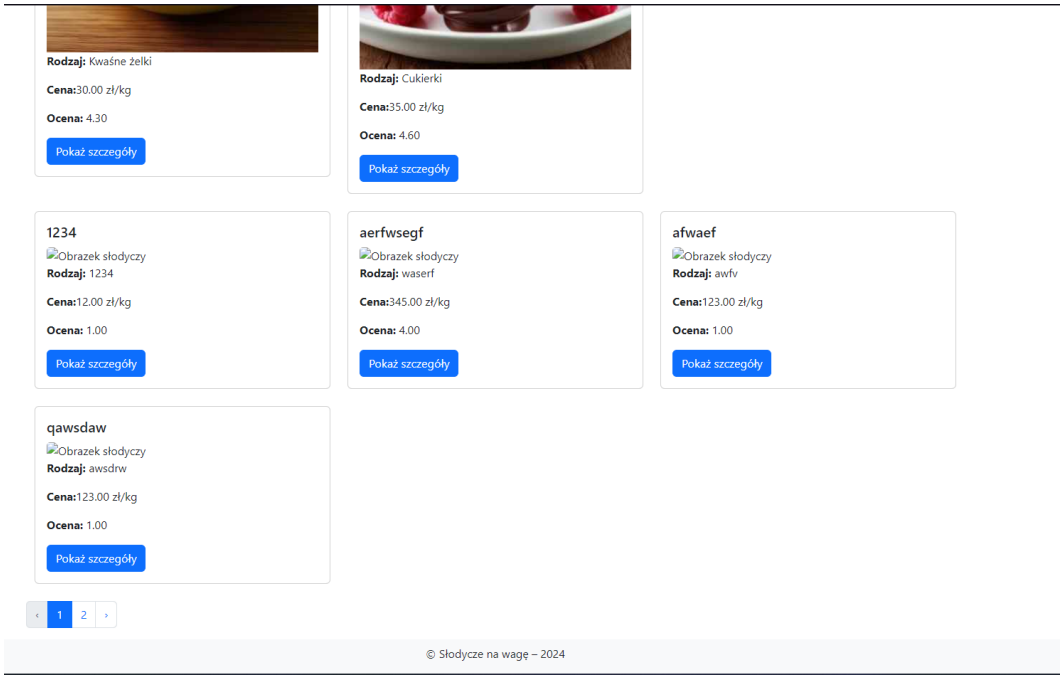
Rysunek 54 Informacje o logice biznesowej dostępne dla użytkownika

- Paginacja i problem n+1

Paginacja została zaimplementowana do projektu przy wyświetlaniu słodyczy.

```
public function index()
{
    return view('candies.index', [
        'candies' => Candy::Paginate(10),

    ]);
}
```

Rysunek 55 Paginacja w funkcji index



Rysunek 56 Wyświetlające się przyciski nawigacyjne, kiedy jest więcej niż 10 produktów

Aby ikonki nawigacyjne wyświetlały się poprawnie należało dodać odpowiedni wpis do AppServiceProvider.

```
});
Paginator::useBootstrap();
```

Rysunek 57 Poprawne wyświetlanie paginacji

Problem n+1 został rozwiązany w następujący sposób:

```
$orders =  Order::with(['candies', 'user'])->get();
```

Rysunek 58 Pobieranie danych zamówień

Metoda with() służy do ładowania powiązanych danych (relacji) w jednym zapytaniu SQL. Każde zamówienie zostanie pobrane wraz z danymi powiązanymi z tabelami candies i users.

```
select * from `orders`

select `candies`.*, `candy_order`.`order_id` as `pivot_order_id`, `candy_order`.`candy_id` as `pivot_candy_id`, `candy_order`.`quantity` as `pivot_quantity`, `candy_order`.`price` as `pivot_price` from
`candies` inner join `candy_order` on `candies`.`id` = `candy_order`.`candy_id` where `candy_order`.`order_id` in (1, 2, 3, 4, 5, 6, 7) and `candies`.`deleted_at` is null

select * from `users` where `users`.`id` in (1, 3, 5, 6)
```

Rysunek 59 Zapytania potrzebne do wyświetlenia zamówień